

above are pure pipes (the pure property is set to true). If a pipe is pure, it will re-execute only the value of the input or the passed parameter are changed. While this sounds reasonable and in alignment with the nature of pipes in general, it is possible that in some cases we need a particular pipe to re-run independently of such a restriction, in this case we can make the pipe 'impure' by setting the property 'pure' to false.

Example: <http://pastebin.com/6smpCpOW>

This code is a use of custom pipes which we will see later in the chapter. A custom pipe countdown is created, setting the pure property to false. It has a timer inside it which runs in every 50 milliseconds and changes the value of a certain property of the pipe.

Create a custom pipe

Custom pipes allow you to create your own pipes and leverage it to perform complex operations on a data. This is the true significance of pipes. It allows you to use your functions directly on a dataset like an operator. Custom pipes are a powerful tool that can be used in a variety of ways, starting from simple domain specific validation rules to pre processing and filtering data sets based on user preferences. Below is an example of creating and using a custom pipe.

hooks

Pipe classes has three hooks.

- constructor(arguments)
- transform(value, arguments)
- onDestroy()

constructor() is the normal constructor of the class, transform() function is the actual method that is called when the pipe is executed, and onDestroy() is called once the pipe is going to be destroyed.

Creating a custom pipe

To get setup, we need to import Pipe and PipeTransform from the Angular 2 core:

```
• import { Pipe, PipeTransform } from '@angular/core';
```

Next, we need to export and decorator our class with the right metadata and also use implements PipeTransform: